

Evaluación Globalizadora A: Levitador Neumático

Bustamante, Fernando

Portas, Nahuel

UNIVERSIDAD TECNOLÓGICA NACIONAL-FACULTAD REGIONAL AVELLANEDA

fernando.miguel.bustamante@gmail.com

nahuluciano@gmail.com

Índice

1. Fundamentación.....	2	8. Esquemático y PCB.....	7
1.1. Problemática.....	2	9. Conclusiones.....	8
1.1.1. Problemática Físico - Mecánica.....	2	10. Referencias.....	8
1.1.2. Problemática de Control e Instrumentación.....	2	ANEXO 1: DIAGRAMA DE CONEXIONES.....	9
1.2. Solución.....	2	ANEXO 2: DIAGRAMA DE TAREAS.....	10
1.2.1. Solución Físico - Mecánica.....	2	ANEXO 3A: VISTA 3D DEL PCB (LADO DE LOS COMPONENTES).....	11
1.2.2. Enfoque de control.....	2	ANEXO 3B: VISTA 3D DEL PCB (LADO DEL COBRE).....	12
1.3. Fundamentación de hardware.....	3	ANEXO 4A: IMPRESO PARA EL PCB (LADO DEL COBRE), ESCALA 1:1.....	13
1.3.1. Parte Físico - Mecánica.....	3	ANEXO 4B: IMPRESO PARA EL PCB (LADO DE COMPONENTES), ESCALA 1:1.....	14
1.3.2. Componentes de Instrumentación y Control.....	3	ANEXO 5: ESQUEMÁTICO.....	15
2. Fundamentación de software.....	3	ANEXO 6: PLACA IMPRESA (LADO DEL COBRE).....	16
3. Objetivos.....	4	ANEXO 6B: PLACA IMPRESA (LADO DE LOS COMPONENTES).....	17
4. Alcance.....	4		
5. Diagrama de conexiones.....	4		
6. Especificaciones de hardware.....	4		
7. Especificaciones de software.....	6		

Abstract

Este documento detalla la implementación de un sistema de control de lazo cerrado para estabilizar la altura de un objeto liviano. El desafío principal es equilibrar la fuerza de sustentación neumática con la

fuerza de la gravedad para mantener el objeto en la altura deseada (setpoint), compensando perturbaciones externas. La solución implementa un controlador PID sobre la plataforma de sistema embebido Raspberry Pi Pico. Para garantizar la concurrencia de procesos críticos y la ejecución

determinista del lazo de control se optó por un sistema operativo de tiempo real (RTOS). Se seleccionó FreeRTOS por su diseño optimizado y su licencia de código abierto. Esta arquitectura de software permite una gestión eficiente de tareas, como la adquisición de datos del sensor y la actuación del sistema, logrando una levitación estable y precisa.

Palabras Claves: Control PID, FreeRTOS, Raspberry Pico, Sistema embebido, Concurrencia, Determinismo.

1. Fundamentación

Un levitador neumático es un dispositivo diseñado para utilizar aire u otro gas a presión y generar una fuerza de sustentación que contrarresta la gravedad permitiendo que un objeto levite de manera estable en el aire sin contacto físico con superficies sólidas. Su funcionamiento de basa en el equilibrio dinámico de tres principios fundamentales:

- Flujo de aire a presión: El levitador expulsa aire a alta velocidad a través de una boquilla o superficie porosa.
- Fuerza de sustentación: El aire crea una zona de baja presión entre el objeto y el levitador (efecto Bernoulli o cojín de aire), generando una fuerza ascendente que contrarresta el peso del objeto.
- Estabilización: Algunos diseños usan sensores y sistemas de retroalimentación para ajustar el flujo de aire y mantener la levitación estable.

1.1. Problemática

Se presentan dos desafíos que deberán abordarse en la etapa de diseño para el proyecto.

1.1.1. Problemática Físico - Mecánica

Esta dimensión requiere de un diseño y armado del sistema de levitación que considere factores como la fuente de flujo de aire del sistema físico, y las propiedades del objeto a levitar (peso, dimensión, forma y material). Para nuestro caso se usará una esfera de poliestireno expandido. Si bien el peso es

reducido, la falta de un sellado adecuado para concentrar el flujo de aire puede generar perturbaciones. Estas perturbaciones representan una carga para el sistema de control de lazo cerrado, dificultando la estabilidad.

1.1.2. Problemática de Control e Instrumentación

El segundo desafío de implementar un sistema de control a lazo cerrado con el objetivo de mantener la estabilidad del objeto en flotación. Esto implica varios puntos críticos:

1. Medición de Distancia: Determinar el método de medición más fiable para medir la distancia entre el objeto y la fuente de aire, así como la selección de un sensor para la medición de la altura, dicho sensor debe contar con un rango y error aceptable.
2. Algoritmo de Corrección: Desarrollar el algoritmo de control que convierta la señal del sensor en una acción de corrección. Si bien el control se realizara con un PID, el desafío reside en la sintonización óptima de las variables K_p , K_d y K_i .

1.2. Solución

La solución propuesta al desafío de la estabilidad se aborda mediante una arquitectura dual:

1.2.1. Solución Físico - Mecánica

Para la mitigación de inestabilidades físico-mecánicas, se propuso el uso de un tubo de acrílico transparente para confinar la levitación, limitando los grados de libertad laterales de la esfera de poliestireno. Adicionalmente, se modificó la superficie inferior del objeto para garantizar que el flujo de aire impacte de manera uniforme, promoviendo una sustentación más estable.

1.2.2. Enfoque de control

La estabilidad dinámica se logra mediante la implementación de un controlador PID (Proporcional-Integral-Derivativo) en un lazo cerrado. El algoritmo se ejecutará sobre la plataforma Raspberry Pi Pico y se programará bajo un RTOS, específicamente FreeRTOS,

para asegurar una ejecución determinista y la correcta gestión de tareas.

1.3. Fundamentación de hardware

La selección de los componentes de hardware se basó en los requisitos de bajo costo, precisión en la medición y la necesidad de un control de actuación rápida y determinista.

1.3.1. Parte Físico - Mecánica

Para asegurar un entorno de levitación estable se se optó por lo siguiente:

1. Objeto a Levitar: Se seleccionó una esfera de poliestireno de 9 gramos y 7.8 cm de diámetro. Su bajo peso minimiza la fuerza de sustentación requerida, haciendo viable el uso de un cooler de baja potencia.

2. Confinamiento: Se utiliza un tubo de acrílico de aproximadamente 50 cm de longitud, con un diámetro ligeramente superior al de la esfera. Su función principal es proporcionar una guía física que limite los movimientos laterales (grados de libertad), simplificando el problema de control a una sola dimensión (la altura).

3. Flujo de Aire: Se incorpora una plancha de acrílico perforada que se coloca entre el cooler y el tubo. El objetivo de esta plancha es homogeneizar el flujo de aire, promoviendo un régimen laminar y reduciendo las turbulencias que podrían desestabilizar la levitación.

4. Actuador: El flujo de aire es proporcionado por un Cooler controlado por PWM. Se elige este actuador por ser una solución de bajo costo y por su capacidad de generar la fuerza de aire suficiente para elevar el objeto liviano, además de que su señal de control PWM de 3.3 V es directamente compatible con la plataforma Pico.

1.3.2. Componentes de Instrumentación y Control

1. Plataforma de Control (Raspberry Pi Pico): Se optó por la Raspberry Pi Pico debido a su arquitectura de doble núcleo (Dual-Core) y su potente SDK C/C++,

que es esencial para la programación eficiente de sistemas embebidos en tiempo real como FreeRTOS.

2. Sensor de Altura (HC-SR04): Para medir la altura entre la fuente de aire y la esfera, se selecciona el sensor de ultrasonido HC-SR04. Su elección se basa en que proporciona una señal digital (no requiere ADC) y cuenta con un rango de medición (3 cm a 400 cm) y un error aceptable para este prototipo.

3. Interfaz de Usuario (Potenciómetro y LCD):

3.1. La selección de los parámetros (setpoint, max y min) se realiza mediante un potenciómetro, lo que requiere utilizar un conversor ADC de la Pico.

3.2. Para la visualización, se utiliza una pantalla LCD 20x4 con conexión I2C, que permite mostrar simultáneamente todos los parámetros clave del sistema.

4. Datalogger y RTC: Para el registro de datos (datalogger) se incluye una memoria SD. La inclusión de un Reloj de Tiempo Real (RTC) que comparte el bus I2C con el LCD garantiza que los registros de eventos incluyan sellos de fecha y hora precisos.

2. Fundamentación de software

Si bien se puede realizar el código usando solamente el SDK C/C++ de la Pico, se optó por el uso de FreeRTOS, por las siguientes razones:

1. Determinismo: Se asegura que la adquisición de datos y el cálculo del PID se realicen en tiempos predecibles y sin *jitter*, minimizando la latencia y la pérdida de información esencial para el control en tiempo real.

2. Concurrencia y Priorización: El código se divide en tareas independientes a las que se asignan prioridades. Esto garantiza que la tarea de control tenga preferencia sobre las tareas menos críticas (como la interfaz de usuario o el datalogger).

3. Gestión de Recursos: Facilita la gestión eficiente de la memoria y la comunicación entre tareas, utilizando primitivas como las **colas (queues)** para la transferencia de datos y los **semáforos mutex** para el

acceso seguro a recursos compartidos (ej. el bus I2C que sera mencionado mas adelante).

Esta estructura modular de software mejora la mantenibilidad y permite futuras optimizaciones sin comprometer la estabilidad del sistema de control.

3. Objetivos

El objetivo principal es diseñar e implementar un levitador neumático estable controlado por un sistema embebido (Raspberry Pi Pico con FreeRTOS), capaz de mantener la posición de una esfera dentro de un margen de error aceptable definido por un *setpoint*.

Pero ademas se fija objetivos mas específicos, como:

1. Control del Setpoint: Implementar la funcionalidad para establecer el setpoint deseado y sus límites de seguridad (setpoint_max y setpoint_min), con una resolución de 1 cm en el rango de operación.

2. Medición y Adquisición de Datos: Asegurar la medición continua y precisa de la altura mediante el sensor ultrasónico HC-SR04 con una frecuencia de muestreo mínima de 10 Hz para la retroalimentación del lazo de control.

3. Estabilidad y Precisión del Control: Implementar y sintonizar el controlador PID para mantener la esfera en el setpoint con un error de estado estacionario no superior a 1 cm.

4. Datalogger y Monitoreo: Desarrollar el software para registrar variables críticas del sistema (fecha, hora, setpoint y medición) en una memoria SD para el análisis de rendimiento.

4. Alcance

El sistema tanto fisico-mecánico como de control fue basado estrictamente en las condiciones físicas específicas del objeto. Por lo tanto, se establece como limitación que la selección de un objeto con las mismas dimensiones, pero diferente material y peso, requerirá de un proceso de re-sintonización completa del PID y una re-evaluación de la potencia mínima del a fuente de aire.

El sistema de control y el diseño mecánico (el tubo) están enfocados en la estabilización vertical (eje Z) sin compensar movimientos laterales (eje X, eje Y). Su entorno de operación es cerrado y controlado, no se contempla la compensación de perturbaciones significativas.

5. Diagrama de conexiones

La conexión de los elementos de hardware se puede ver en el ANEXO 1, donde se pueden observar el flujo de datos entre los mismos.

6. Especificaciones de hardware

En base al ANEXO 1 que es el diagrama en conexiones del hardware se indicara cada uno de los elementos junto a sus características.

1. Plataforma Raspberry Pico: Se utilizo el modelo Raspberry Pi Pico 2 (RP2350).

- Memoria flash 4MB
- Memoria RAM 512Kb
- Dos módulos SPI
- Dos módulos I2C
- Dos módulos UART
- 3 entradas ADC
- PWM
- Alimentación 5V por el pin VSYS

2. HC-SR04: Este sensor es uno de los mas accesibles del mercado comparado con los infrarrojos, su bajo costo lo hace ideal para esta aplicación.

- Rango de operación 2cm a 400cm
- Angulo efectivo menor a 15°
- Resolución de 0.3 cm
- Ancho de pulso para el disparo: 10useg
- Alimentación: 5V
- Corriente de trabajo 15 mA

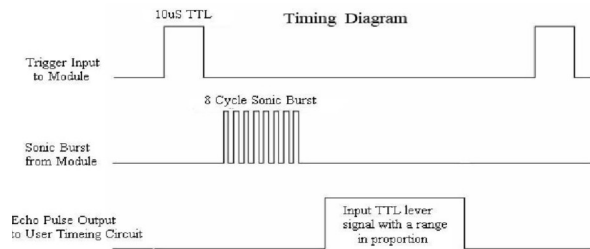


Figura 1. Diagrama temporal del HC-SR04



Figura 2. Sensor HC-SR04

Pin	Descripción
VCC	Alimenticio de 5V
Trig	Pin para el pulso de disparo
Echo	Pin de salida indicando recepción del rebote
GND	Conexión a Tierra

3. Modulo RTC: Reloj de tiempo real RTC DS3231.

- Tensión de alimentación: 5V
- Cuenta: segundos, minutos, horas, días, meses, años
- Batería: tipo botón de 5V para mantener la fecha
- Comunicación I2C



Figura 3. Modulo RTC

Pin	Descripción
GND	Conexiona a Tierra
VCC	Alimentación
SDA	Pin de entrada/salida de datos para I2C
SCL	Pin de clock para I2C
SQW	Permite configurar una onda cuadrada o una señal de interrupción por alarma
32K	Salida de un reloj de referencia estable

4. Fuente de alimentación: Todo el circuito será alimentado por una fuente switching de 12Vx2A. De la cual se extraen los 3.3V para el ADC y el modulo I2C, y los 5V para alimentar los sensores y otros módulos. Se emplean dos reguladores LM7805 y LM317 ajustable a 3.3V.

5. Modulo de Memoria SD: Memoria para el datalogger, modelo MH-SD

- Tensión de alimentación: 5V 0 3.3V
- Capacidad máxima: 2 TB
- Comunicación SPI

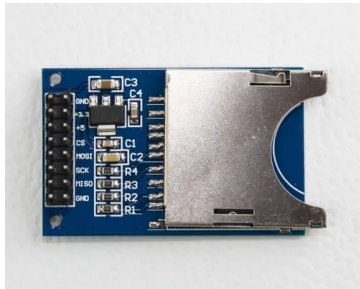


Figura 4. Módulo SD

Pin	Descripción
GND	Conexión a Tierra
3.3	Alimentación de 3,3V
5	Alimentación de 5V
CS	Habilita/deshabilita la comunicación con el módulo
MOSI	Datos de salida del microcontrolador (Maestro), habilita la SD
SCK	Línea de reloj que sincroniza la transferencia de datos
MISO	Datos de entrada hacia el microcontrolador (Esclavo) desde la SD
GND	Conexión a Tierra

6. Fuente de aire: Se usa un cooler de PC de fabricación Arctic.

- Alimentación: 12V
- Tensión de PWM: 3.3V
- Tacómetro: si
- Dimensiones en cm: 80(L)x80(W)x25(H)



Figura 5. Cooler P8 Arctic

7. Visualización: LCD de 20x4 con comunicación I2C



Figura 6. LCD 20x4 con I2C

Pin	Descripción
GND	Conexión a Tierra
VCC	Alimentación de 5V
SDA	Entrada/Salida de datos I2C
SCL	Entrada de clock del I2C

8. Adicionales: elemento de uso común.

- Potenciómetro de 10K para seleccionar el Setpoint
- Leds para indicar las alertas de máximo y mínimo (Rojos)

- Botón normal abierto para la selección de opciones

9. Fuente de alimentación: Para alimentar los componentes, se emplea una fuente de alimentación switching de 12 V (para el cooler). Sin embargo, para la circuitería lógica (que requiere 5V y 3.3 V), se utilizan reguladores de voltaje.

El regulador LM7805 es un Regulador de Tensión Lineal (no switching), seleccionado por su sencillez de implementación y bajo ruido, aunque su eficiencia energética es menor que la de una fuente conmutada. Se complementa con un LM317 (regulador ajustable) para la obtención precisa de los 3.3 V requeridos por la Raspberry Pi Pico y sus periféricos.

7. Especificaciones de software

EL firmware cuenta con un total de 10 tareas, de las cuales 2 son para testeo del botón de opciones y 1 para testear el sensor de medición. La forma en que se relacionan las tareas se pueden ver en el ANEXO 2.

La tarea *task_init* es la de mayor prioridad y se encarga de inicializar los elementos de la Raspberry Pi Pico y luego se elimina. Esta tarea puede reemplazarse por las funciones de inicialización.

La tarea *task_Setpoint* es la que configura la altura de flotación y los límites, de esta tarea se envía información, por medio de colas, a las siguientes tareas:

task_pid (cola queue_pid)
task_guradiana_lcd (cola queue_setpoint)
task_guradiana_sd (cola queue_sd)
task_guradiana_leds (cola queue_leds)

Las colas permiten que se pueda transferir información entre tareas de manera segura y directa, eliminando las variables globales cuyos datos de pueden corromper si son accedidas por varias tareas.

Los semáforos son un forma de administrar el acceso a un recurso (semáforo mutex) logrando proteger el acceso al recurso I2C, compartido por el LCD y el RTC.

Se utiliza otro tipo, llamado semáforo binario, el cual me permitirá activar la tarea *task_guardiana_sd*

para guardar el seteo en una primera instancia. Pero también para guardar las alertas de de máximo y mínimo.

Si bien ambos son semáforos su funcionamiento es muy diferente, en el caso del mutex solamente la tarea que lo tomo lo puede devolver, mientras que en el caso del binario si una tarea lo toma otra tarea cualquiera lo puede ceder.

A parte de los aspectos de FreeRTOS debemos resaltar la forma adquirir los datos para el seteo. Para esta acción se utiliza un potenciómetro conectado al ADC. Pero el ADC de la Raspberry Pi Pico tiene una resolución de 0 a 3,3V y se necesitan alturas no tensiones. Para solventar este inconveniente se uso la ecuación siguiente, en la cual se multiplica por un factor de adecuación, la conversión del valor del ADC a tensión.

$$H_{cm} = N \times \frac{3.3}{4095} \times F_a$$

N: Valor entero que corresponde a una tensión convertida por el ADC

F_a : Factor de adecuación, en nuestro caso usamos 10.

De esta manera convierto una magnitud eléctrica en una magnitud física.

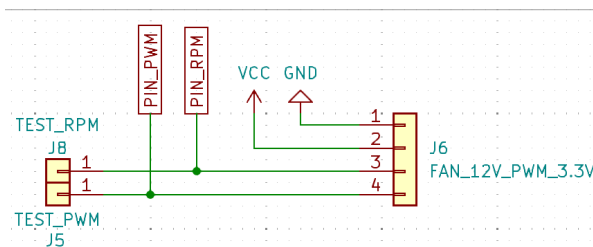
8. Esquemático y PCB

Para la realización del diseño electrónico del circuito (esquemático y PCB) se utilizó el software KiCAD versión 8.0. El diseño se estructuro de manera modular en el esquemático para facilitar la comprensión y la depuración. La división comprende los siguientes bloques funcionales:

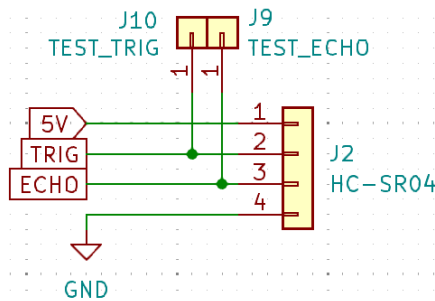
- Bloque de Sensores y Módulos: Conexión del sensor HC-SR04, conexión del modulo RTC, coenxnion del modulo SD.
- Bloque de Alimentación: Contiene los reguladores de tensión LM7805 para suministrar los 5V, el regulador LM317 para suministrar los 3.3V.

- Bloque de Banderas: Dedicado a los elementos de interfaz de usuario; LCD, leds y botón de opciones.

En el diseño del PCB se fijaron estratégicamente puntos de prueba (test point), los cuales se pueden reconocer por la nomenclatura **TEST_XXX**. En total so 4 puntos de prueba; dos para el sensor HC-SR04 en los pines Trig y Echo, dos para el cooler pudiendo medir el PWM y la lectura del tacómetro.



Test point para el cooler



Test point para HC-SR04

9. Conclusiones

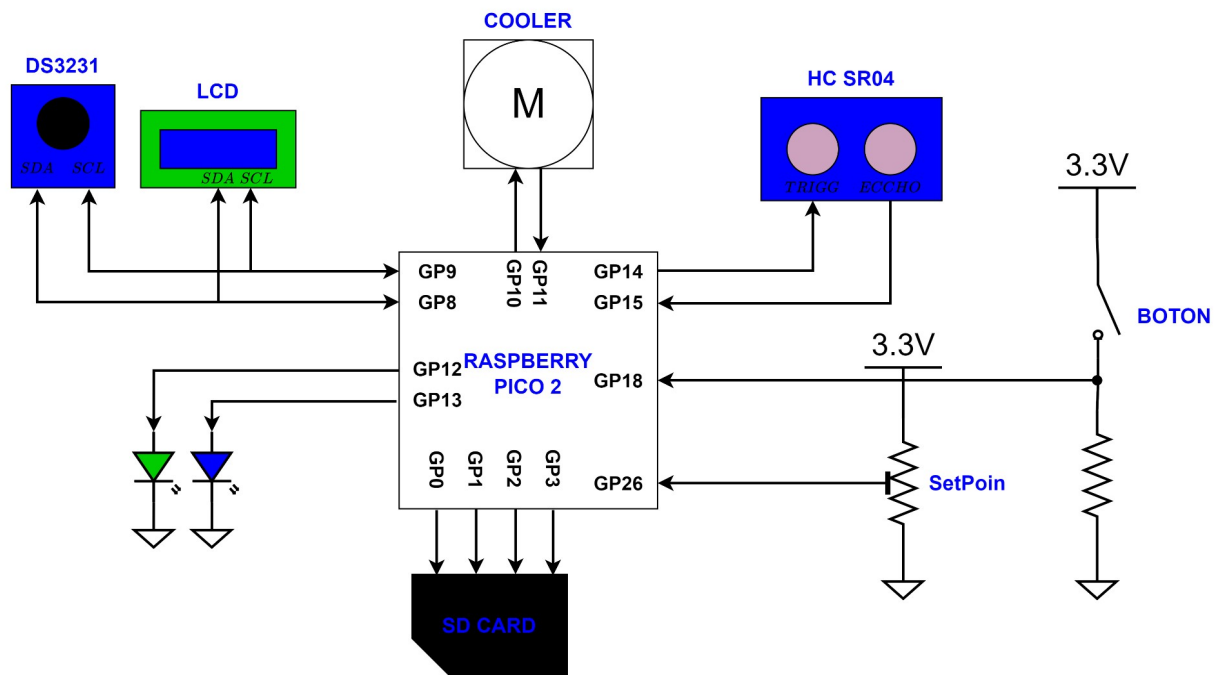
En base al diseño de la parte Fisico-Mecanica y electrónica podemos concluir lo siguiente:

- La construcción del elemento Fisico-Mecanico debe ser lo mas hermética posible a fin de evitar fugas de aire.
- Se debe proveer una superficie plana para que el flujo de aire golpe de manera uniforme.
- Se debe colocar un plancha perforada a fin de tener un flujo laminar.
- Los tres apartados anteriores permitirán reducir el esfuerzo realizado por el controlador.
- La plataforma Raspberry con el SDK C/C++ permitió una programación mas fluida en conjunto con el kernel de FreeRTOS, cuyas APIs son directas y muy intuitivas.
- El uso de semáforo mutex fue la elección correcta para asegurar una comunicación limpia de I2C, en la cual dos dispositivos distintos usaban el mismo I2C. Con el mutex se permitió que cada dispositivo tomara el recurso el tiempo que fuera necesario.
- Las colas fueron una herramienta importante para transferir información de forma segura y directa entre tareas, con su utilización evitamos el uso de datos corruptos de variables globales.

10. Referencias

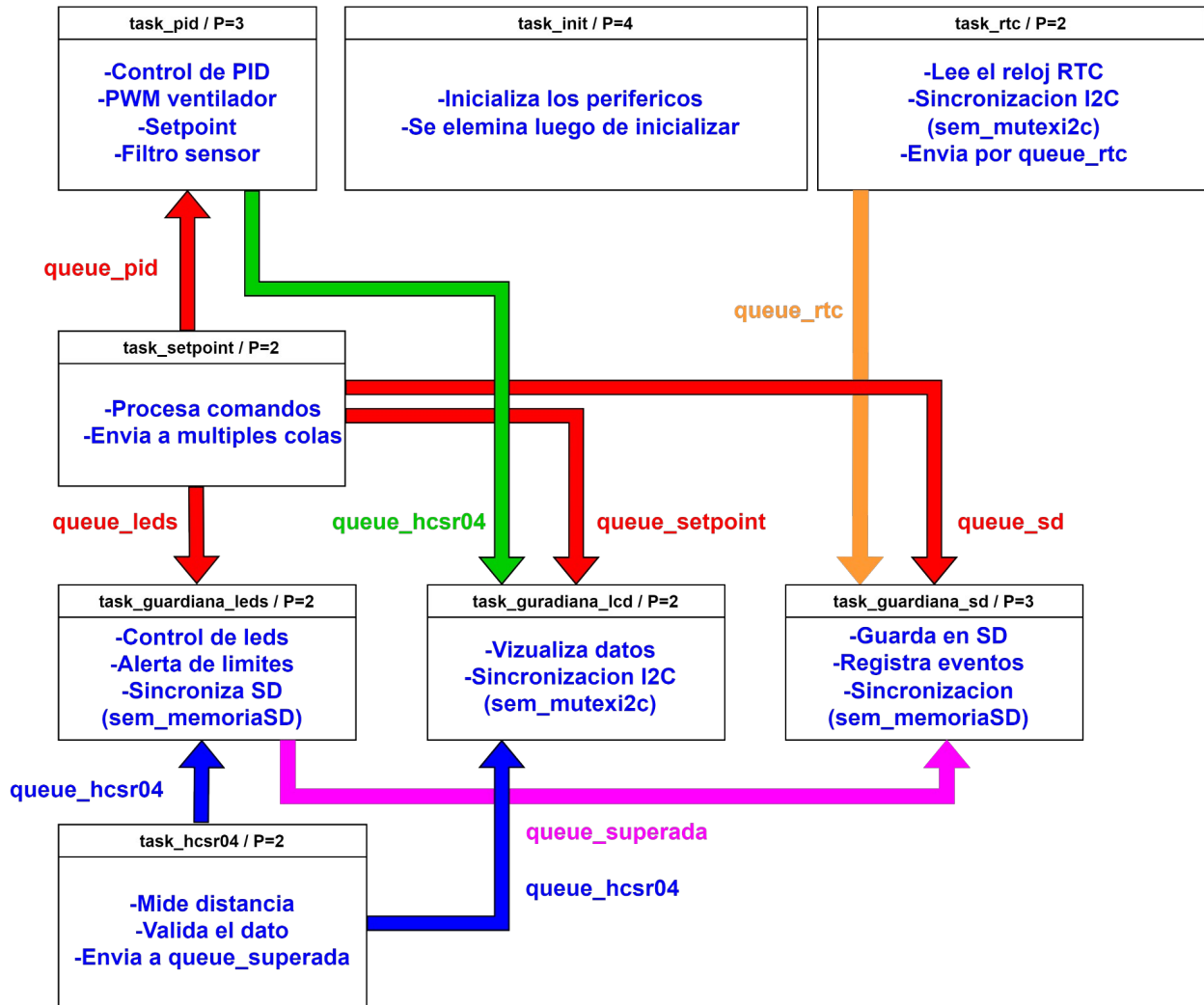
- [1] FreeRTOS, Pagina Oficial, <https://www.freertos.org/>
- [2] Plataforma Raspberry, Pagina Oficial, <https://www.raspberrypi.com/documentation/microcontroller/s/pico-series.html>, https://www.raspberrypi.com/documentation/microcontrollers/c_sdk.html
- [3] Hojas de datos: <https://www.alldatasheet.es/datasheet-pdf/pdf/1132203/ETC2/HC-SR04.html>, <https://support.arctic.de/p8max>

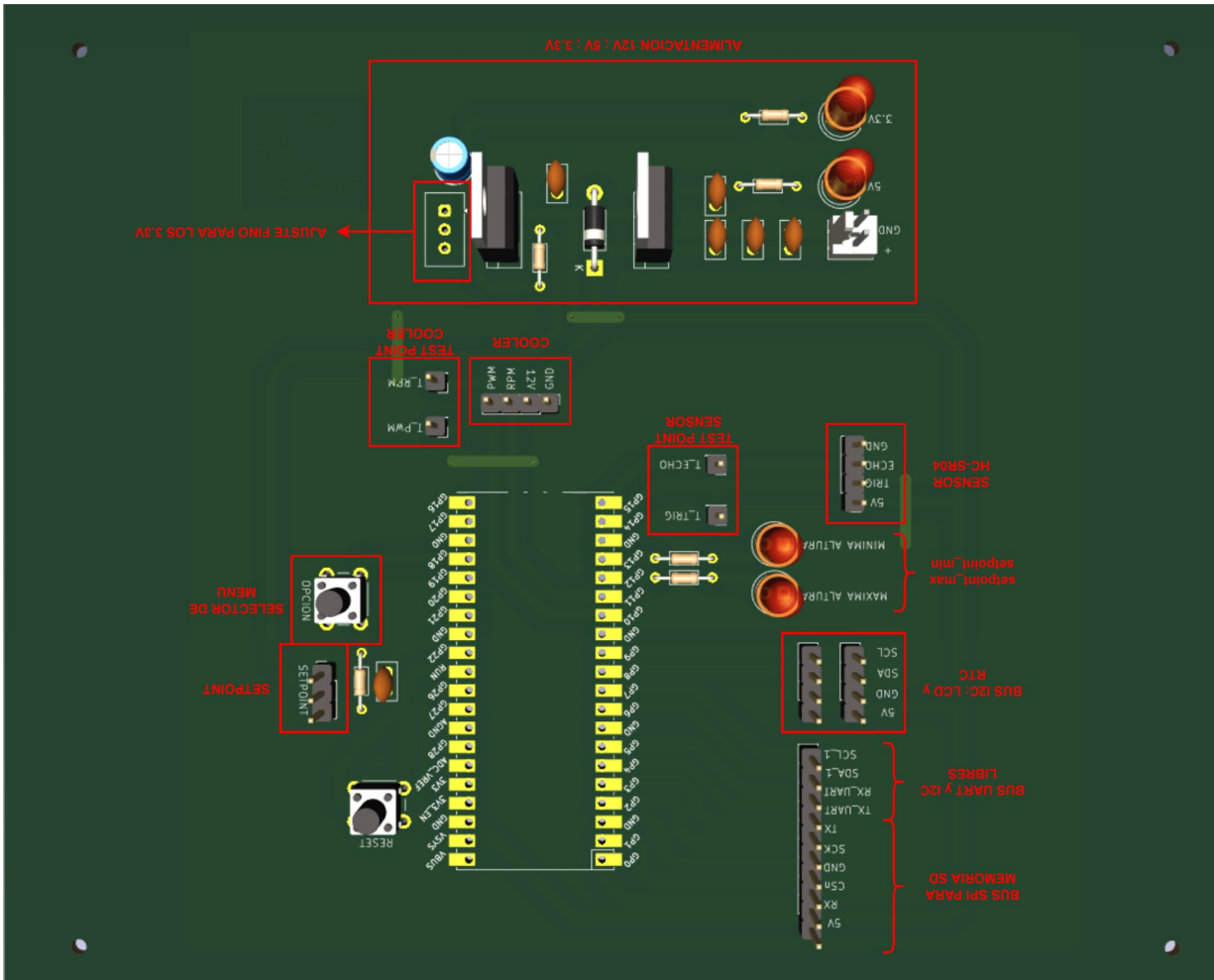
ANEXO 1: DIAGRAMA DE CONEXIONES

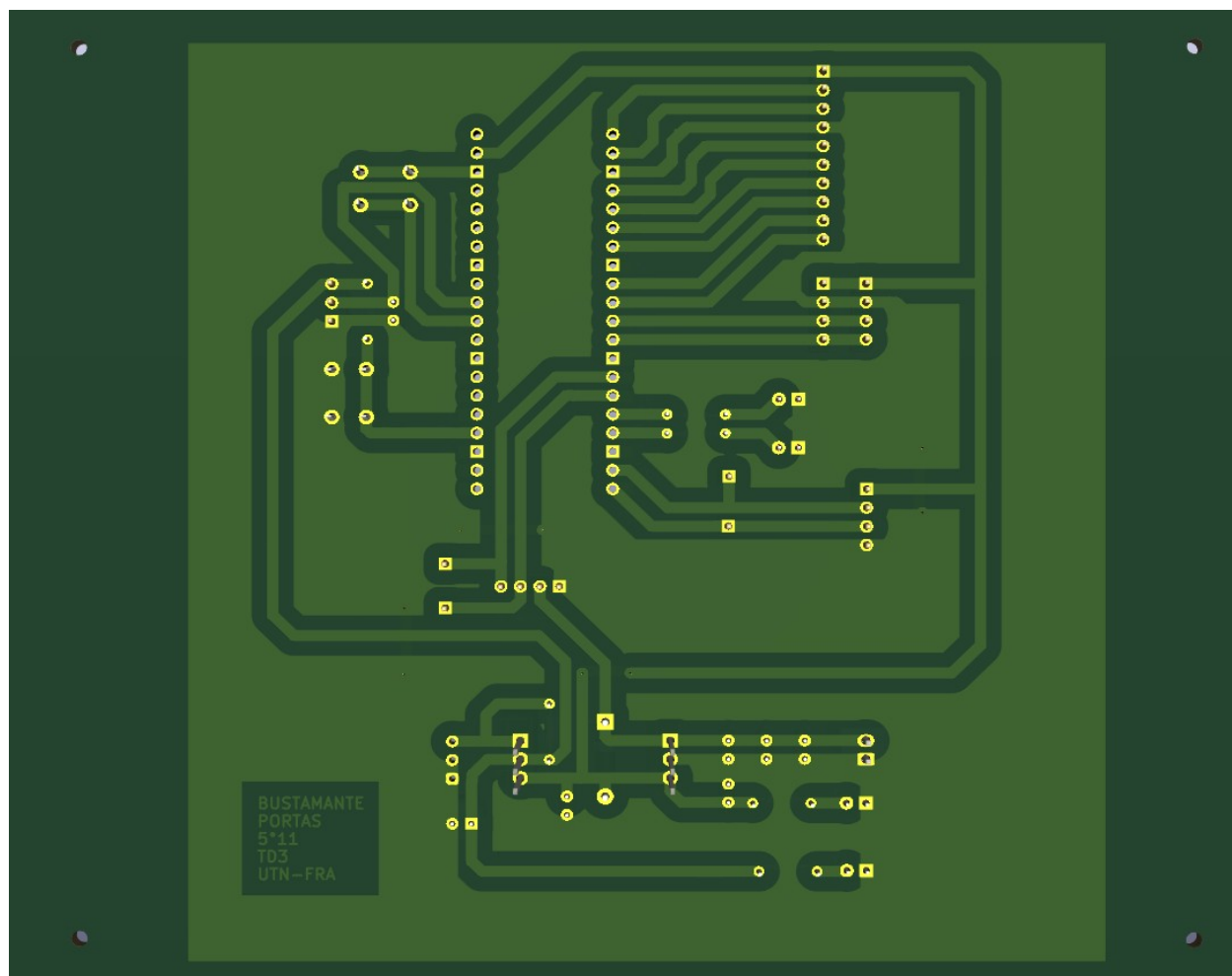


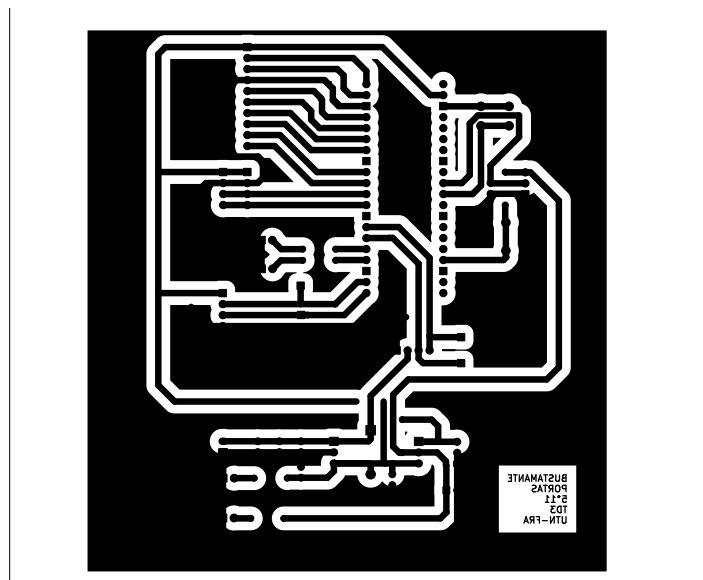
INFORMACION DE PINES	
GP8 ; GP9	SDA, SCL
GP15 ; GP16	TRIGG, ECHHO
GP10, GP11	PWM ; RPM
GP12, GP13	MAXIMO ; MINIMO
GP0, GP1, GP2, GP3	MISO, CSn, SCK, MOSI
GP18	Pin 24, BOTON

ANEXO 2: DIAGRAMA DE TAREAS

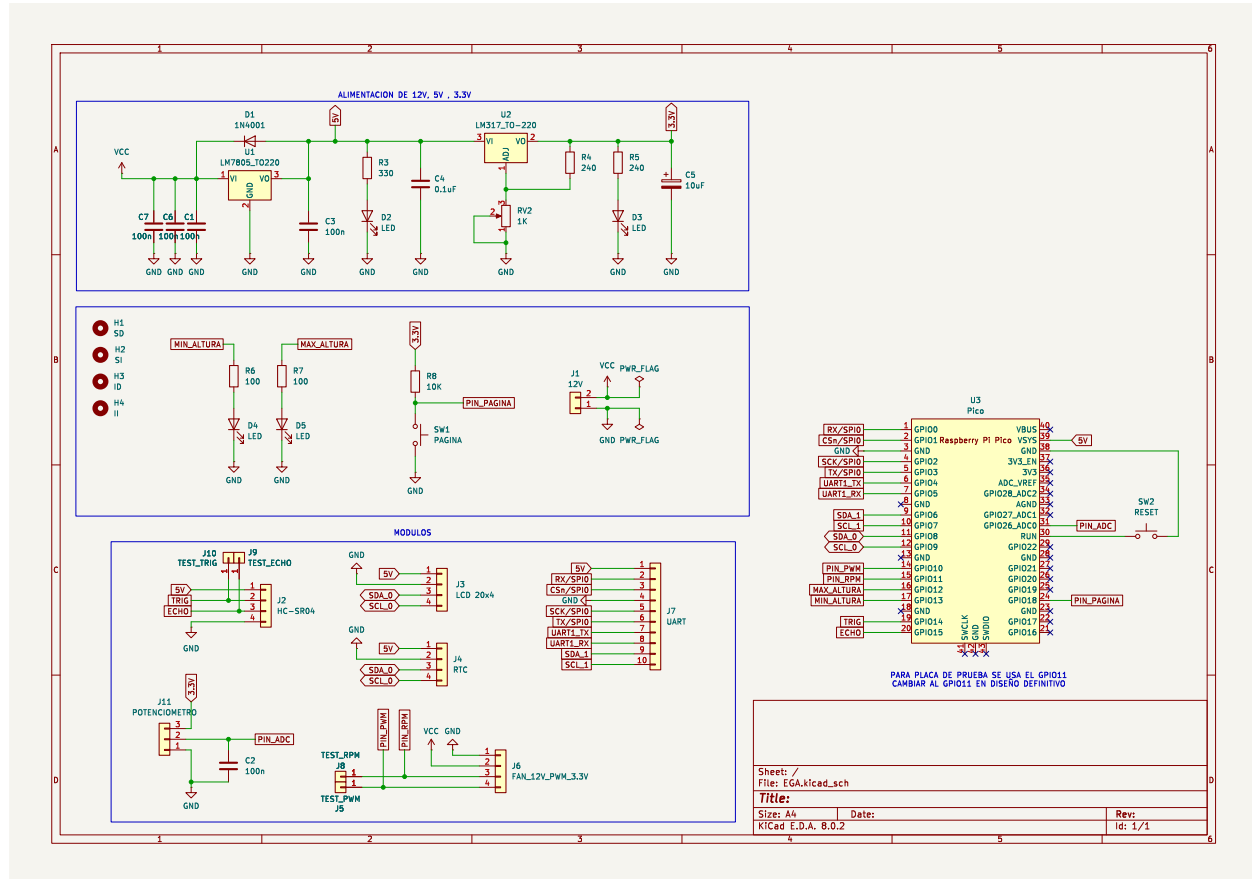


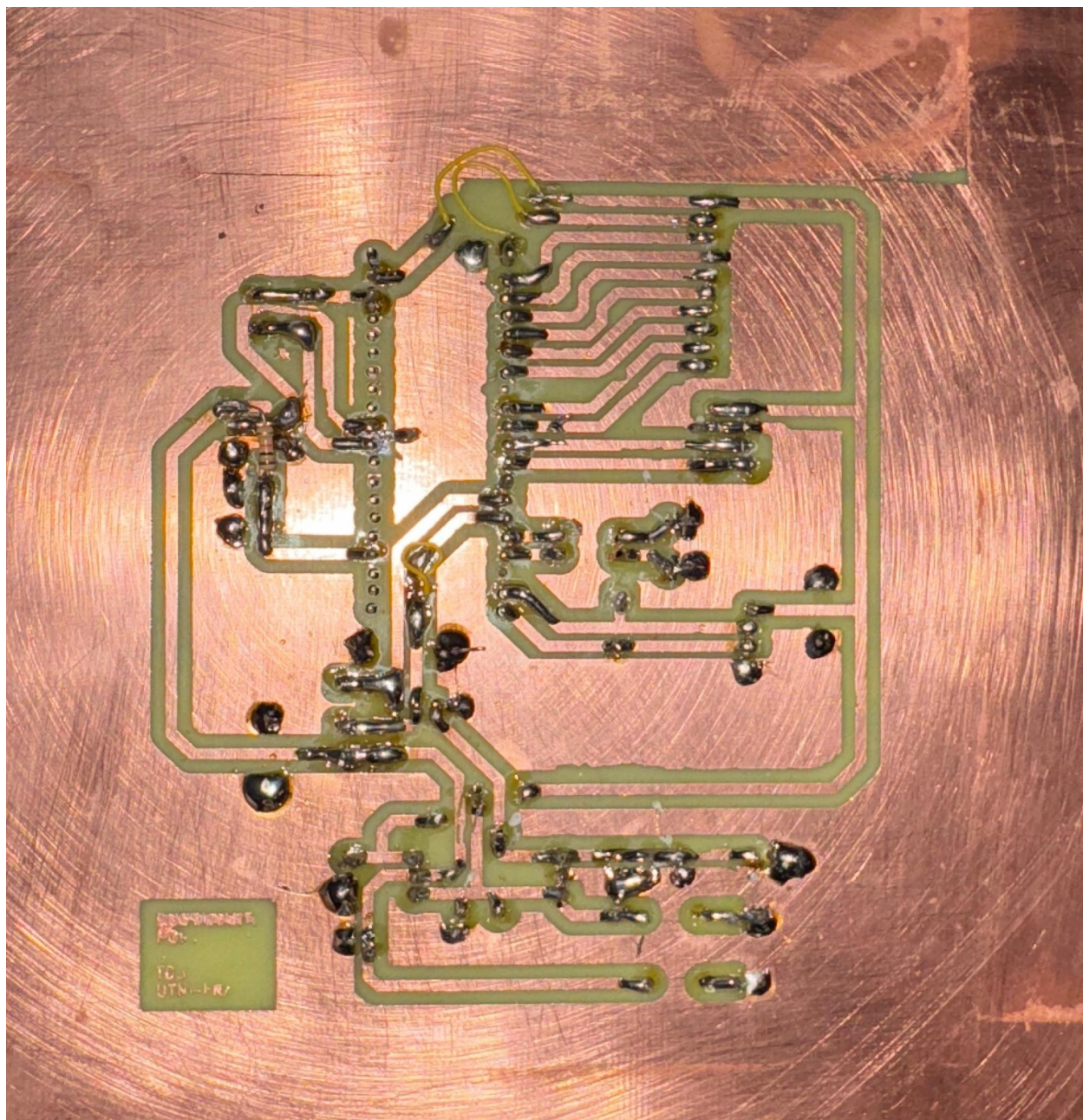
ANEXO 3A: VISTA 3D DEL PCB (LADO DE LOS COMPONENTES)

ANEXO 3B: VISTA 3D DEL PCB (LADO DEL COBRE)

ANEXO 4A: IMPRESO PARA EL PCB (LADO DEL COBRE), ESCALA 1:1

ANEXO 5: ESQUEMÁTICO



ANEXO 6: PLACA IMPRESA (LADO DEL COBRE)

ANEXO 6B: PLACA IMPRESA (LADO DE LOS COMPONENTES)